

Étude numérique des oscillations d'un cylindre : l'influence du confinement

Jules Chamoy & Lofty Gauthier

Magistère de Physique Fondamentale d'Orsay

Projet informatique 2025

jules.chamoy@universite-paris-saclay.fr • lofty.gauthier@universite-paris-saclay.fr



Contexte

Nous faisons partie de l'équipe de Paris-Saclay pour le *French Physicist Tournament*. Pour l'expérience dont nous nous occupons, nous étudions les oscillations d'un cylindre tiré vers le haut lorsqu'il est plongé dans un fluide confiné entre deux plaques.

Sans confinement, un cylindre plongé dans un fluide en translation latérale subit une instabilité de **Bénard–Von Karman**, produisant des oscillations de faible amplitude et haute fréquence. Avec confinement entre deux plaques parallèles au mouvement, une **nouvelle instabilité** apparaît, donnant des oscillations de grande amplitude et faible fréquence. Cette instabilité a été découverte au laboratoire FAST par Benoît Semin, lors de l'étude de fibres traversant des milieux poreux.

Nous cherchons alors à simuler un cylindre confiné entre deux plaques, se déplaçant à vitesse constante, en restreignant la situation à une coupe 2D du cylindre pour des raisons de facilité de simulation, et car on peut considérer une invariance selon l'axe longitudinal du cylindre.

Modèle physique

Notre situation est un problème de mécanique des fluides ; on cherche alors à simuler les équations de Navier-Stokes :

$$\rho \left[\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right] = \rho \mathbf{g} - \nabla P + \eta \nabla^2 \mathbf{u}$$

avec $\mathbf{u}$ le champ de vitesse, ρ la densité du fluide, ∇P le gradient de pression à l'intérieur du fluide, $\mathbf{g}$ le champ d'accélération de pesanteur, ici orienté selon z , tel que $\mathbf{g} = -g\mathbf{e}_z$, et η le coefficient de viscosité dynamique du fluide.

De plus, on se place dans une situation de fluide réel et incompressible, ainsi on a $\eta \neq 0$ et $\nabla \cdot \mathbf{u} = 0$. Dans la pratique, ces conditions imposent une densité constante, et nos conditions aux limites seront :

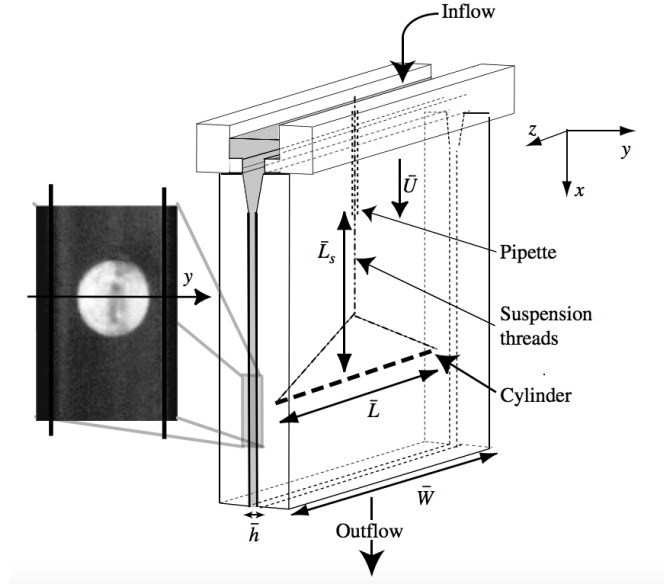


Figure 1: Schéma du montage expérimental tiré de [1]

- $\mathbf{u}_{\text{paroi}} = \mathbf{0}$ car les parois sont immobiles,
- $\mathbf{u}_{\text{cylindre}} = \mathbf{U}_{\text{cylindre}}$ où $\mathbf{U}_{\text{cylindre}}$ est la vitesse du cylindre dans le fluide.

On gardera notre cylindre fixe selon l'axe horizontal pour faciliter l'approche numérique en premier lieu, et il remontera selon l'axe z , contre la gravité, à une vitesse constante $\mathbf{U}_{\text{cylindre}} = U\mathbf{e}_z$.

Enfin, pour l'approche numérique, nous avons opté pour la projection de Chorin, dont le principal avantage est de découpler le calcul du champ de vitesse $\mathbf{u}$ et du champ de pression P .

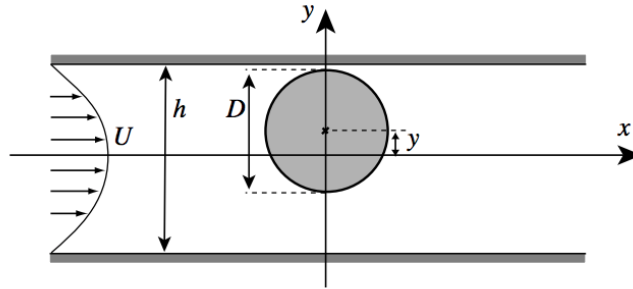


Figure 2: Vue 2D du problème utilisée pour la simulation tiré de [1]

Discretisation numérique

Pour réaliser cette simulation, on utilisera la méthode des Volumes Finis, qui consiste en une discrétisation de l'espace et du temps, avec une discrétisation temporelle variable.

On introduit les notations suivantes :

$$\mathbf{u}_{(i,j)}^n ; P_{(i,j)}^n$$

où n représente le n -ième pas de temps, tandis que le couple (i, j) permet de se repérer dans le quadrillage 2D.

Vitesse intermédiaire

Dans la projection de Chorin, on définit une vitesse intermédiaire $\mathbf{u}^*$ telle que :

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = -(\mathbf{u}^n \cdot \nabla) \mathbf{u}^n + \mathbf{g} + \nu \nabla^2 \mathbf{u}^n \quad (1)$$

où $\nu = \frac{\eta}{\rho}$ est la viscosité cinématique et Δt le pas de temps de la discrétisation temporelle.

On a pour la dérivée temporelle de $\mathbf{u}$:

$$\frac{\partial \mathbf{u}}{\partial t} = \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t}$$

et ainsi on peut trouver la relation reliant le gradient de pression à $\mathbf{u}^*$:

$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla P^{n+1} \quad (2)$$

Or, le fluide est incompressible, soit $\nabla \cdot \mathbf{u}^{n+1} = 0$; donc si on applique la divergence à l'équation précédente, on obtient l'équation de Poisson suivante :

$$\nabla^2 P^{n+1} = \frac{\rho}{\Delta t} \nabla \cdot \mathbf{u}^* \quad (3)$$

On trouvera ensuite le champ de vitesse au temps $n + 1$ en inversant l'équation 1.

Condition CFL

Pour les volumes finis, on choisira une grille cartésienne de dimensions $L_x \times L_z$, avec $N_x \times N_z$ cellules ; les pas d'espace sont donnés par $dl = L/N$. En Volumes Finis, pour des raisons de stabilité numérique, on doit respecter la condition CFL (Courant-Friedrichs-Lewy), qui impose qu'à chaque pas de temps, les flux sortants de chaque cellule ne parcourent pas plus d'**une case**.

En effet, la méthode des Volumes Finis consiste à calculer à chaque pas de temps les contributions des flux sortants et entrants dans chaque cellule. Si les flux parcourent plus d'une case en un pas de temps, des instabilités numériques apparaissent car il y a perte et/ou apparition de données fantômes.

Si les différents flux ont une vitesse de propagation c_i , alors le pas de temps doit respecter :

$$\Delta t \leq \min \left(\frac{dl}{c_i} \right) \quad \forall i$$

Dans notre situation, en régime incompressible, il n'y a pas d'onde de compression ; les seules vitesses de propagation viennent des vitesses u_x et u_z , et de la diffusion visqueuse. Pour une diffusion visqueuse 2D, la stabilité impose $\Delta t \leq \frac{dl_x^2}{2\nu}$ et $\Delta t \leq \frac{dl_z^2}{4\nu}$, ce qui donne la condition CFL :

$$\Delta t \leq \min \left(\frac{dl_x^2}{4\nu}, \frac{dl_z^2}{4\nu}, \frac{dl_x}{u_x^{\max}}, \frac{dl_z}{u_z^{\max}} \right) \quad (4)$$

Schémas de dérivation numérique

Nous venons de voir comment résoudre temporellement ces équations afin d'obtenir notre simulation évolutive. Cependant, celles-ci font intervenir des laplaciens, des divergences et des gradients ; or numériquement il existe plusieurs manières de dériver.

Soit une fonction $f(x)$ dans un espace 1D discrétisé en cellules discrètes espacées de Δx , alors on peut dériver :

- en arrière : $\frac{\partial f}{\partial x} = \frac{f_i - f_{i-1}}{\Delta x}$
- en avant : $\frac{\partial f}{\partial x} = \frac{f_{i+1} - f_i}{\Delta x}$
- au centre : $\frac{\partial f}{\partial x} = \frac{f_{i+1} - f_{i-1}}{2\Delta x}$

Ces schémas n'ont pas tous la même stabilité ni la même précision selon la grandeur dérivée.

Le laplacien est naturellement centré ; en 2D on l'écrit :

$$\nabla^2 f(x, y) = \frac{f_{(i+1,j)} + f_{(i-1,j)} - 2f_{(i,j)}}{\Delta x^2} + \frac{f_{(i,j+1)} + f_{(i,j-1)} - 2f_{(i,j)}}{\Delta y^2} \quad (5)$$

avec Δx et Δy les pas spatiaux selon x et y .

Pour la pression, on utilisera toujours des dérivées centrées car le gradient est une grandeur indépendante de l'orientation de l'espace, donc la dérivation doit également être isotrope. Les divergences seront également centrées car les dérivées centrées sont d'ordre 2 et donc plus précises.

Schéma upwind

En revanche, le **terme convectif** nécessite un autre schéma de dérivation pour des raisons de stabilité. En effet, une dérivée centrée peut engendrer des instabilités ; on utilise alors un schéma dit **upwind**, très répandu en analyse numérique vectorielle.

Son fonctionnement est simple : selon le sens du vecteur, on applique une dérivée avant ou arrière. Dans notre cas :

$$\frac{\partial f}{\partial x}(x, y) = \begin{cases} \frac{f_{i+1,j} - f_{i,j}}{\Delta x} & \text{si } f < 0 \\ \frac{f_{i,j} - f_{i-1,j}}{\Delta x} & \text{si } f > 0 \end{cases} \quad (6)$$

Résolution multigrille

Il nous reste à résoudre la pression, régie par l'équation de Poisson 3. Pour ce faire, il existe plusieurs schémas. Le premier un peu évolué se nomme SOR (Successive Over-Relaxation). Il s'agit d'une amélioration de la méthode de Gauss-Seidel pour la résolution de systèmes numériques $A\phi = b$, A étant la matrice de l'application linéaire. Pour un espace discret, le laplacien peut être représenté comme une matrice carrée, et l'équation de Poisson devient un système linéaire.

On note $\phi^{(k)}$ la k -ième itération de ϕ ; pour $\nabla^2 \phi = f$, on a :

$$\phi_{i,j}^{(k+1)} = (1 - \omega)\phi_{i,j}^{(k-1)} + \omega\phi_{i,j}^{(k),GS} \quad (7)$$

avec

$$\phi_{i,j}^{(k),GS} = \left(\frac{\phi_{i-1,j}^{(k)} + \phi_{i+1,j}^{(k-1)}}{\Delta x^2} + \frac{\phi_{i,j-1}^{(k)} + \phi_{i,j+1}^{(k-1)}}{\Delta y^2} - f_{i,j} \right) \left(\frac{2}{\Delta x^{-2} + \Delta y^{-2}} \right)^{-1}$$

qui est l'itération de Gauss-Seidel. On doit régler la valeur de ω pour que la méthode converge le plus rapidement possible sans diverger. Cette méthode est parmi les plus rapides, cependant pour de grandes grilles elle n'est pas suffisamment rapide. On va donc se servir de la méthode de résolution par **multigrilles**.

La méthode par sur-relaxation est mal adaptée aux grandes grilles car les erreurs prennent trop d'itérations pour se propager. L'idée est alors de passer par des grilles plus petites. On peut résumer cette méthode en 5 étapes :

- **Lissage** : application de quelques itérations SOR (moins d'une dizaine) sur la grille d'origine de taille $N_x \times N_z$ et calcul du résidu $r_{i,j} = f_{i,j} - \nabla^2 \phi_{i,j}$,
- **Restriction** : calcul par interpolation du résidu sur une grille plus grossière de taille $(N_x/2) \times (N_z/2)$. Ce faisant, les erreurs se rapprochent et deviennent plus simples à traiter par SOR,
- **Correction** : résolution de $\nabla^2 e = r$ sur la grille grossière, e étant la correction apportée,
- **Prolongation** : par interpolation dans le sens inverse, on transfère la correction vers des grilles plus fines jusqu'à revenir à la taille initiale,
- **Post-lissage** : application de quelques itérations SOR sur le résultat obtenu.

La variante la plus simple, employée ici, est le V-cycle : on effectue la restriction jusqu'à atteindre une grille de taille 4×4 , on calcule la correction, puis on fait la prolongation jusqu'à revenir à la taille originelle.

Pour cela, on utilisera une récursion dans l'exécution du V-cycle, avec des tailles de système $N_x, N_y = 2^\alpha$ pour simplifier les calculs. Le nombre de récursions optimal est $\log_2(\min(N_x, N_z)) - 2$ car on veut que les côtés de la grille la plus grossière aient une taille minimale de 4 cases.

Enfin, on applique cet algorithme N_{iter} fois afin de faire converger le résultat.

Étapes de calcul

Après avoir présenté notre modèle de résolution, on peut décrire l'algorithme de résolution.

Tout d'abord, on initialise notre système :

- Nombre de cellules et taille du système dans les deux dimensions d'espace,
- Position et taille du cylindre,

- Vitesse de l'écoulement,
- Viscosité et densité du fluide,
- Temps maximal de la simulation.

On choisit les conditions initiales de telle sorte qu'on change de référentiel : depuis celui du fluide (immobile, dans lequel on tire le cylindre) vers le référentiel du cylindre (immobile dans un fluide en mouvement).

Avec ces conditions, on calcule le premier pas de temps grâce à la condition CFL, qui nous donne Δt .

Ensuite, on calcule $\mathbf{u}^* = \mathbf{u}^n + \Delta t(\nu \nabla^2 \mathbf{u}^n + \mathbf{g} - (\mathbf{u}^n \cdot \nabla) \mathbf{u}^n)$, car c'est grâce à ce champ qu'on peut calculer le terme source de l'équation de Poisson 3 à résoudre pour la pression.

On résout ensuite l'équation de Poisson avec le schéma multigrille, après avoir calculé la divergence de $\mathbf{u}^*$. Le nombre maximal d'itérations du V-cycle et la tolérance sur l'erreur maximale sont déterminés au préalable.

Ayant la vitesse et la pression dans le fluide à un instant n , on peut calculer la vitesse à l'instant $n + 1$ avec :

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla P^{n+1}$$

On applique ensuite les conditions aux limites du système sur la vitesse. Les conditions aux limites pour la pression (conditions de Neumann) sont déjà incluses dans la résolution de l'équation de Poisson.

Enfin, on fait boucler l'algorithme depuis la condition CFL jusqu'à atteindre le temps final voulu.

Structure de classes pour la résolution numérique

Pour réaliser ces étapes, nous avons eu besoin de 4 classes différentes :

1. **Grille.h** : contient toutes les informations relatives à la structure, sa taille dans les deux directions, l'obstacle, etc.
2. **Champ.h** : permet de stocker et d'utiliser un tableau 1D de taille $N_x \times N_z$ comme si on utilisait un tableau 2D de taille N_x par N_z ,
3. **Solveur.h** : contient toutes les fonctions relatives aux méthodes de dérivation et aux schémas de résolution SOR et multigrilles,
4. **Liquide.h** : contient les paramètres physiques du fluide, les champs physiques, et les fonctions membres permettant la réalisation de l'algorithme.

Grille

Cette classe a pour unique but de contenir les informations structurales : ses nombres de cellules N_x et N_z , ses dimensions physiques L_x et L_z , et les pas spatiaux associés dl_x et dl_z . Elle possède également les informations relatives à l'obstacle.

Elle stocke la position du cylindre selon x et z , son rayon, et surtout un **tableau de booléens renseignant sur la position du cylindre**. Ce tableau, de même taille que la grille, est calculé dès l'initialisation d'un objet de type *Grille*. Pour chaque cellule, il renvoie **true** si cette cellule est une cellule solide, et **false** sinon.

L'intérêt de ce tableau est de permettre de savoir en permanence si on se situe sur une case fluide ou solide, ce qui est utile pour les conditions aux limites, la dérivation numérique, et pour éviter des calculs inutiles dans les cases solides.

Champ.h

Dans notre simulation, on utilise trois champs différents, P , $\mathbf{u}$, et $\mathbf{u}^*$, soit cinq tableaux (deux dimensions d'espace), parcourus en permanence dans les calculs. Afin de gagner du temps de calcul, on utilise des tableaux 1D de taille $N_x \times N_z$ plutôt que des tableaux 2D de taille N_x par N_z : bien que la mémoire occupée soit identique, l'accès est légèrement plus rapide pour les tableaux 1D, ce qui peut représenter un gain considérable pour de très grandes grilles.

L'intérêt de cette classe est de permettre de manipuler des tableaux 1D en raisonnant avec des coordonnées 2D (x, z) .

Solveur.h

Dans cette simulation, on utilise des méthodes générales, comme des opérateurs de dérivation discrète et des méthodes de résolution d'équations (ici l'équation de Poisson), qui sont donc indépendantes de la situation physique.

Il nous a semblé pertinent de les rassembler dans un espace de noms annexe ne contenant que des fonctions : *Solveur* est un *namespace* et non un type.

Liquide.h

Nous avons enfin besoin d'une dernière classe qui regroupe l'intégralité de nos paramètres et des caractéristiques du fluide. Un objet de type *Liquide* contient un objet *Grille*, la viscosité cinématique ν et la densité ρ , ainsi que cinq objets *Champ* : P , u_x , u_z , u_x^* et u_z^* .

Elle contient également toutes les fonctions membres relatives à la mise en place de l'algorithme, comme le calcul de Δt pour la condition CFL, le calcul des composantes de $\mathbf{u}^*$, de la pression P , et l'application des conditions aux limites.

Nous avons décidé d'utiliser une unique classe permettant d'accéder à tout ce qui est nécessaire, car pour des raisons de clarté nous souhaitons ne réaliser dans **Main.cpp** que l'initialisation d'un unique objet auquel on applique les fonctions de modification.

Classes auxiliaires

En complément des classes principales décrites précédemment, trois classes auxiliaires assurent respectivement la visualisation en temps réel, l'export des résultats et la configuration du programme.

Simulation.h

La classe **Simulation** est le point d'entrée de l'affichage. Elle prend en référence un objet *Liquide* et orchestre deux tâches concurrentes via deux threads : le thread principal gère la fenêtre **GLFW** et le rendu **OpenGL**, tandis qu'un thread secondaire (**simulationLoop**) effectue les calculs physiques. La synchronisation entre les deux est assurée par un mutex protégeant un tampon partagé (**renderBuffer**), dans lequel les données physiques sont copiées à chaque pas de temps. Un drapeau atomique **newFrameReady** signale au thread de rendu qu'une nouvelle image est disponible.

L'affichage repose sur deux textures **OpenGL** : **solidTexture**, chargée une seule fois au démarrage et représentant le masque de l'obstacle solide, et **dataTexture**, mise à jour dynamiquement à chaque frame avec le champ physique choisi parmi **ux**, **uy**, **p** ou **u_norm**. La méthode publique **run()** lance l'ensemble du processus et bloque jusqu'à la fermeture de la fenêtre.

Export.h

Le namespace **Export** fournit deux fonctions permettant de sauvegarder les résultats de la simulation en fin d'exécution. La fonction **champCSV()** exporte un champ scalaire quelconque dans un fichier **.csv** au format **x, y, valeur**, une ligne par cellule de la grille. La fonction **exportTout()** appelle simplement **champCSV()** pour chacun des trois champs physiques u_x , u_y et P , produisant trois fichiers séparés dans le dossier de sortie spécifié. Ces fichiers peuvent ensuite être exploités par des outils externes comme Python ou Gnuplot pour une analyse post-traitement.

Config.h

La classe **Config** est un lecteur de fichiers **.ini**. Elle charge et parse un fichier de configuration organisé en sections (**[section]**) et en paires **clé = valeur**, en ignorant les commentaires introduits par **;** ou **#**. Elle expose ensuite trois accesseurs typés — **getString()**, **getInt()** et **getDouble()** — qui retournent une valeur par défaut si la clé demandée est absente. Cela permet de modifier tous les paramètres physiques et numériques de la simulation sans recompiler le projet, et garantit un comportement cohérent même si le fichier de configuration est incomplet.

Structure des fichiers

Le projet est organisé en plusieurs dossiers distincts afin de séparer les dépendances du code source. Le répertoire **SRC/** contient l'ensemble du code source en C++, incluant les classes principales telles que la simulation, le solveur, la gestion des shaders et des entités physiques. Le dossier **SHADERS/** regroupe les fichiers GLSL utilisés pour le rendu graphique. Les répertoires **build/** et **DEP/** sont dédiés respectivement à la compilation et aux dépendances externes (**GLAD**, **GLFW**). Le dossier **diagrams/** contient les schémas UML du projet, tandis que **output/** est utilisé pour les résultats générés (fichiers **.csv**). Enfin, les fichiers à la racine (comme **CMakeLists.txt**, **config.ini** et **README.md**) assurent la configuration et la documentation du projet.


```
crazy_cylinder /
|-- SRC/
|-- SHADERS/
|-- diagrams/
|-- build/
|-- DEP/
|-- output/
'-- fichiers de configuration
```

Figure 3: Structure principale du projet

CMake

Le projet utilise **CMake** comme système de configuration et de compilation afin de garantir une construction portable et modulaire. Le fichier `CMakeLists.txt` principal, situé à la racine, définit la version minimale requise, le standard C++ utilisé (C++20) ainsi que les dépendances nécessaires, notamment **OpenGL**, **Threads** et **OpenMP**. Il assure également la copie automatique des shaders et du fichier de configuration dans le dossier de compilation. Les sous-modules du projet sont organisés via `add_subdirectory`, en particulier le dossier `SRC/` pour le code source et `DEP/GLFW` pour la bibliothèque graphique externe. La compilation s'effectue depuis la racine du projet en deux étapes :

```
cmake -B build
cmake --build build
```

Cette approche permet de séparer les fichiers générés du code source.

Dépendances

Le projet repose sur plusieurs dépendances et nécessite **CMake** en version 3.23 ou supérieure. La bibliothèque **GLFW**, incluse dans le répertoire `DEP/GLFW/`, est utilisée pour la création de la fenêtre d'affichage ainsi que pour la gestion des événements clavier, souris et redimensionnement. Le chargement des fonctions **OpenGL** est assuré par **GLAD**, également fourni localement dans `DEP/GLAD/`. Le rendu graphique repose sur **OpenGL**, qui constitue le cur de l'affichage visuel de la simulation. Le projet exploite également **OpenMP** pour le parallélisme sur processeur. Enfin, le support des threads POSIX est assuré via **pthread**, intégré automatiquement par **CMake** grâce à `Threads::Threads`, afin de gérer l'exécution concurrente des tâches (**OpenGL** et calculs physiques).

OpenGL

La simulation est affichée en temps réel dans une fenêtre **OpenGL** pendant toute la durée du calcul. L'architecture de l'application repose sur une exécution multithreadée organisée autour de deux threads principaux. Le thread principal est chargé de la gestion de la fenêtre **GLFW**, du rendu graphique ainsi que du traitement des événements utilisateur, notamment les entrées clavier. Il s'exécute à chaque image sans bloquer l'affichage

afin de garantir une visualisation fluide. En parallèle, un thread secondaire, nommé `simulationLoop`, effectue les calculs physiques liés à la simulation CFD. Entre deux pas de temps, les données calculées sont copiées rapidement dans un tampon partagé protégé par un mutex, assurant ainsi une synchronisation sûre entre calcul et affichage.

L'appui sur la touche **Échap** permet de fermer la fenêtre et d'arrêter proprement les deux threads.

Shaders

L'affichage de la simulation repose sur un pipeline `OpenGL` simple utilisant un quadrilatère plein écran sur lequel sont appliqués des shaders personnalisés. Le vertex shader (`grid.vert`) a pour rôle de transmettre directement les coordonnées des sommets à l'écran, tout en interpolant les coordonnées de texture associées. Il ne réalise aucune transformation géométrique complexe : chaque fragment correspond ainsi directement à une cellule de la grille de simulation.

Le fragment shader (`grid.frag`) est responsable de la visualisation des données physiques. Il échantillonne deux textures : une première texture `uData`, contenant un champ scalaire normalisé entre 0 et 1 (vitesse, pression, etc.), et une seconde `uSolid`, représentant le masque des obstacles solides. Les cellules solides sont détectées via cette seconde texture : leur intérieur est affiché en sombre, tandis que leur contour est mis en évidence en blanc grâce à une détection de gradient locale. Pour les cellules fluides, la valeur du champ est convertie en couleur via une palette centrée autour de 0,5 (bleu → noir → rouge), permettant de visualiser les variations du champ physique.

Du côté de `Simulation.cpp`, la fonction `render()` met à jour dynamiquement la texture contenant les données de simulation. Les valeurs calculées sont d'abord normalisées symétriquement afin de conserver les variations positives et négatives, puis converties en entiers sur 8 bits avant d'être transférées vers le GPU. À chaque image, les textures sont liées aux unités correspondantes, le shader est activé, puis un quadrilatère couvrant l'écran est dessiné. Chaque pixel affiché correspond ainsi à une cellule de la grille, offrant une représentation directe et en temps réel de l'état de la simulation.

Configuration et paramètres initiaux

L'ensemble des paramètres physiques et numériques de la simulation est centralisé dans le fichier `config.ini`, situé à la racine du projet. Ce fichier est automatiquement copié dans le dossier de compilation par `CMake`, ce qui permet au programme d'y accéder directement à l'exécution. Les paramètres y sont organisés en plusieurs sections logiques : `[grille]` pour la discrétisation spatiale du domaine, `[fluide]` pour les propriétés physiques du fluide, `[cylindre]` pour la définition de l'obstacle solide, `[simulation]` pour les paramètres numériques du solveur et du temps final, ainsi que `[export]` pour la gestion des fichiers de sortie. Cette organisation facilite la modification rapide des conditions de simulation sans intervenir dans le code source, et permet de changer les paramètres sans recompiler le projet. De plus, chaque clé possède une valeur par défaut définie directement dans le

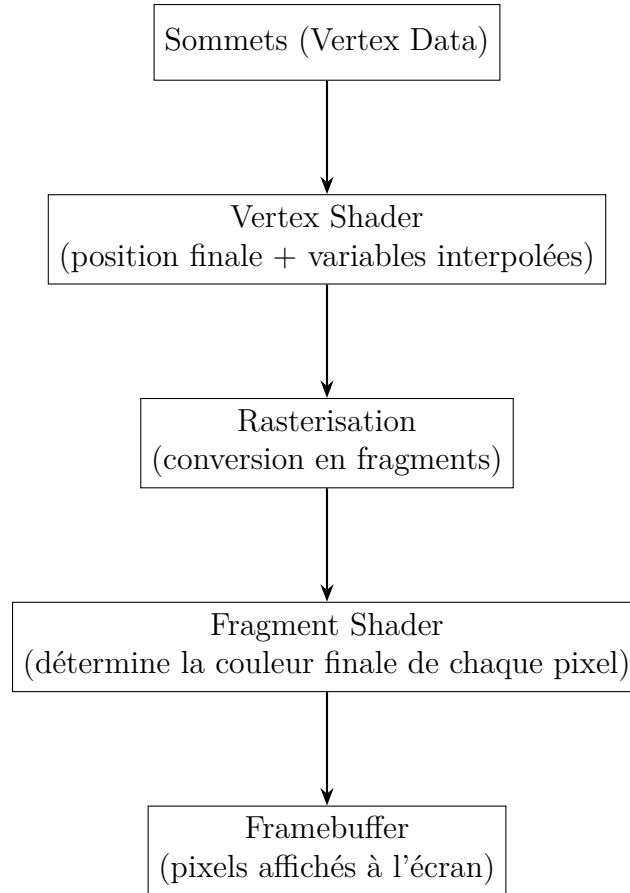


Figure 4: Pipeline OpenGL simplifié. Le fragment shader calcule la couleur finale de chaque pixel.

programme, ce qui garantit le fonctionnement de la simulation même en cas d'absence partielle de certaines entrées dans le fichier de configuration.

Observations et résultats attendus

D'après la littérature, les oscillations apparaissent dans un régime précis caractérisé par le nombre de Reynolds Re et le rapport de densité ρ_s (cf. figure 5). Nous cherchons à retrouver numériquement la même plage d'apparition de l'instabilité de confinement (CIV).

Nous souhaitons caractériser l'instabilité en étudiant le mouvement du cylindre sous différents paramètres : vitesse de remontée, nature du fluide, degré de confinement. Les champs de pression et de vitesse (cf. figure 6) seront comparés aux résultats expérimentaux pour identifier l'origine physique de l'instabilité.

Premiers résultats

Nous n'avons malheureusement pas pu avancer suffisamment pour réaliser les études comparatives souhaitées, en revanche nous avons obtenu des résultats probants et on observe plusieurs phénomènes.

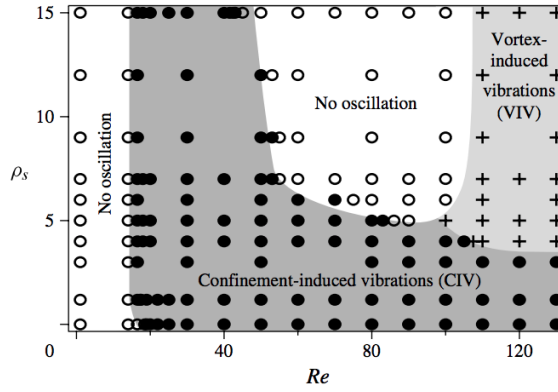


FIGURE 14. Phase diagram of the behaviour of the cylinder for different Re and ρ_s as obtained from numerical simulations: cylinder stable in the midplane of the walls (○); CIV oscillations (●); VIV oscillations (+). The grey shades are qualitative guides for the eye.

Figure 5: Diagramme de phase du comportement du cylindre en fonction de Re et ρ_s tiré de [1]

Tout d'abord, la situation physique est bien représentée car pour le cylindre fixe, on observe un écoulement attendu intuitivement et même qualitativement, dont le comportement varie selon le nombre de Reynolds.

En revanche, des divergences numériques peuvent apparaître, notamment lorsque des vortex de faible diamètre se forment. En effet, les vortex de faible diamètre sont régis par des pressions et vitesses bien plus élevées que dans le reste du fluide et, si le pas spatial est trop grand, la résolution de l'équation de Poisson converge extrêmement lentement, voire diverge, ce qui en plus de fausser les résultats rend les calculs beaucoup plus longs.

Voici un de nos résultats, à $T = 0,27$ s, pour de l'eau, avec une taille spatiale de $1,0 \text{ m} \times 0,5 \text{ m}$, pour une vitesse $U = 1 \text{ cm s}^{-1}$. Le pavage spatial est de 512×512 cellules.

On peut y observer une situation réelle de mécanique des fluides, avec même l'apparition de couches limites à l'arrière du cylindre.

Limites et améliorations

Nous avons donc une simulation de mécanique des fluides avec obstacle complexe fonctionnelle. Cependant, nous n'avons pas atteint notre objectif final. En effet, pour modéliser un cylindre mobile selon l'axe x , il faudrait calculer les forces dues aux contraintes de cisaillement et de pression exercées sur le cylindre, puis déplacer celui-ci en fonction des forces projetées selon x .

De plus, au niveau de l'affichage, on pourrait afficher plusieurs fenêtres lors d'une exécution du programme (pour la pression et les vitesses simultanément, au lieu d'une seule actuellement), et enregistrer les vidéos avec OpenGL.

Enfin, comme mentionné plus haut, nous sommes très dépendants du nombre de cellules pour la stabilité numérique. Plus il y en a, plus la stabilité est assurée, mais plus le temps d'exécution est grand et ce au carré : si on multiplie par λ le nombre de cases dans chaque direction, on multiplie par λ^2 le nombre total de cellules. De plus, plus il y a de cellules

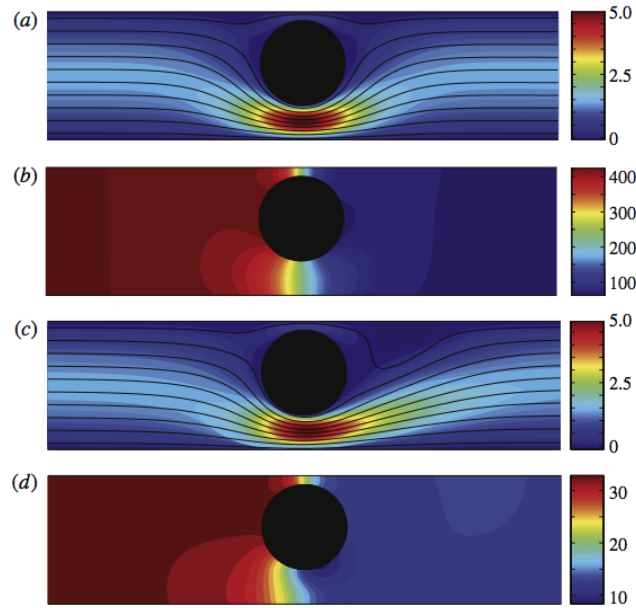


FIGURE 15. (Colour online) Numerical simulation of the flow (from left to right) around a fixed cylinder at (a,b) $Re = 1$ and (c,d) $Re = 20$. In (a) and (c) the colour bar (greyscale) represents the velocity component v_x ; and full lines are streamlines. In (b) and (d) the colour bar (greyscale) represents the pressure field.

Figure 6: Simulation numérique du champ de vitesse et de pression autour du cylindre pour $Re = 1$ et $Re = 20$ tiré de [1]

pour une taille physique L fixe, plus le pas spatial dl diminue, et donc le pas de temps imposé par la condition CFL est plus petit. On voit alors qu'une légère augmentation du nombre de cellules entraîne une grande augmentation du temps d'exécution. Ce lien peut se révéler pénible lorsqu'on a besoin de stabilité numérique, et il serait intéressant de réduire la dépendance du temps d'exécution au nombre de cellules.

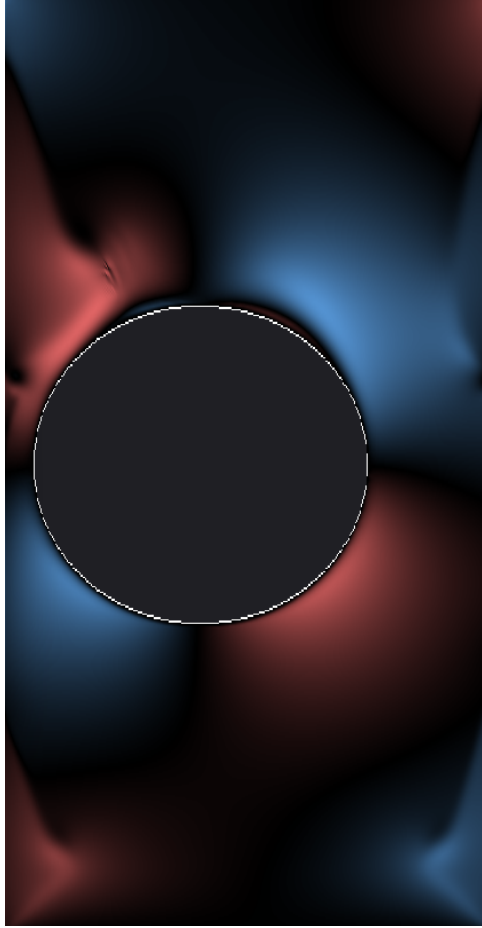


Figure 7: Vitesse u_x du fluide à un instant $T \approx 0,27$ s de la simulation, rouge pour les vitesses positives, bleu pour les vitesses négatives. La situation est inversée, la gravité allant du bas vers le haut et le cylindre allant vers le bas.

References

- [1] B.Semin, A.Decoene, J-P. Hulin, M.François, and H. Auradou. New oscillatory instability of a confined cylinder in a flow below the vortex shedding threshold. *Fluid Mechanics Journal*, 690:345–365, 2012.
- [2] M. V. D’Angelo, J-P Hulin M. Cachile, and H. Auradou. Sedimentation and fluttering of a cylinder in a confined liquid. *Fluid Mechanics Journal*, 2017.
- [3] M. Cachile J.-P. Hulin L. Gianorio, M.V. D’Angelo and H. Auradou. Influence of confinement on the oscillations of a free cylinder in a viscous flow. *Fluid Mechanics Journal*, 2014.

Le diagramme suivant présente l'architecture orientée objet du projet, avec les relations entre les classes principales.

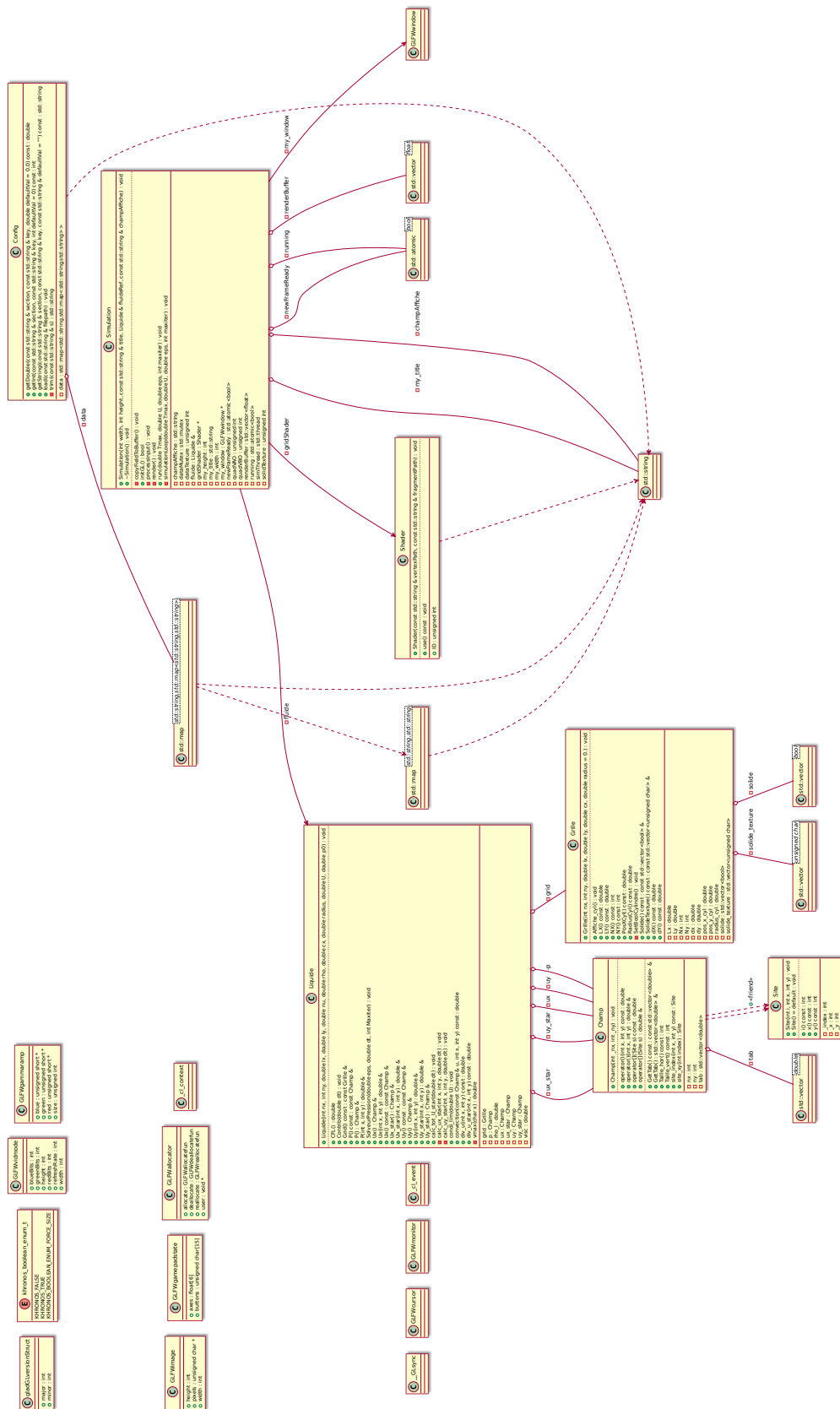


Figure 8: Diagramme UML des classes du projet